

PlaceReady - Smarter Placement Journey

MINI PROJECT REPORT

Submitted By

Nishanthi S (312425205139)

II Year

B. Tech Information Technology

TABLE OF CONTENTS

Sl. No	Contents	Page No
1.	Introduction	1
2.	Problem Statement	1
3.	Scope/Application of the Project	2
4.	Advantages of the Project	2
5.	System Requirements	3
6.	Project Modules	3
7.	Working of the Project	3
8.	Algorithms Used	3
9.	Project Code	4
	database.py	4
	main.py	6
10.	Output Screenshot	11
11.	Conclusion	12
12.	Future Enhancements	12
13.	References	13

1. Introduction

PlaceReady is a Python desktop application designed to help students prepare for placement activities by assessing their skills and organizing practice tasks. The application uses Tkinter for its graphical user interface and SQLite for persistent data storage.

The dashboard introduces four skill areas: Coding, SQL, Aptitude, and Communication. Users can start a skill assessment, view assessment history, maintain practice tasks, and view recommendations based on their latest saved assessment results.

Assessment questions are multiple-choice questions. When an assessment is completed, the application saves the skill category, score, total number of questions, and date/time to a local SQLite database. Users can review previous attempts after closing and reopening the application.

PlaceReady is an individual mini project and a learning tool for practising Python functions, conditional statements, loops, GUI programming, and database operations.

2. Problem Statement

Students preparing for placements need to practise multiple skills and monitor their progress. Keeping track of practice activities and assessment scores manually can be inconvenient, and it may be difficult to remember earlier results or identify areas that need more attention.

A simple desktop-based application is needed to provide skill-based assessments, store assessment results, display assessment history, organize practice tasks, and offer basic recommendations. PlaceReady addresses this need through a graphical interface and a local SQLite database.

3. Scope/Application of the Project

1. Placement Preparation

- Students can practise basic Coding, SQL, Aptitude, and Communication questions.
- Saved scores provide a simple indication of performance in each assessment.

2. Skill Assessment

- The user selects a skill category and answers the multiple-choice questions.
- The final score and total number of questions are saved for later review.

3. Practice Task Management

- Users can add practice tasks, mark them completed or incomplete, and delete a selected task.
- Tasks remain stored in the database between application sessions.

4. Progress Review and Recommendations

- Assessment history displays the skill, score, and date/time of each saved assessment.
- Recommendations are based on the latest saved result for each skill category.

5. Learning Application

- The project demonstrates desktop GUI programming and database operations in Python.

4. Advantages of the Project

1. Easy to Use

A graphical interface provides buttons and separate windows for assessments, history, tasks, and recommendations.

2. Organized Skill Practice

Questions are grouped into four skill categories, allowing focused practice.

3. Persistent Storage

Assessment records and practice tasks are stored in SQLite and can be retrieved in later sessions.

4. Progress Tracking

The history window displays previous scores and timestamps.

5. Task Management

Practice tasks can be added, marked complete, and deleted.

6. Basic Personalized Guidance

Recommendations use the latest score available for each skill category to suggest a priority level.

7. Low Setup Complexity

The project uses Tkinter and Python's built-in sqlite3 module, without requiring a separate database server.

5. System Requirements

Requirement	Details
Hardware	Desktop or laptop computer; 2 GB RAM or more recommended.
Operating System	Windows, Linux, or macOS with Python/Tk support.
Programming Language	Python 3.
GUI Library	Tkinter and ttk.
Database	SQLite using Python's built-in sqlite3 module.
Editor	VS Code, IDLE, or another Python editor.
Database File	placeready.db, used by the application.

No separate database server is required because SQLite stores the application's records in a local database file.

6. Project Modules

1. Dashboard Module

Displays the PlaceReady title, introductory text, four skill cards, and buttons to access the main features.

2. Skill Assessment Module

Displays the question bank for the selected category, collects answers, calculates the score, and saves the result.

3. Assessment History Module

Retrieves saved assessment records and displays skill, score, and date/time in a table.

4. Clear History Module

Asks for confirmation before deleting assessment records and refreshes the table. Practice tasks are stored separately and are not deleted by this feature.

5. Practice Tasks Module

Supports adding tasks, toggling completion status, displaying saved tasks, and deleting a selected task.

6. Recommendation Module

Finds the latest saved assessment for each skill category, calculates its percentage, and displays a corresponding priority message and advice.

7. Database Module

Creates the assessments and tasks tables and provides functions to save, retrieve, update, and delete records.

7. Working of the Project

1. Launching the Application

The main Python file creates the Tkinter window, initializes database tables, and displays the dashboard.

2. Starting an Assessment

The user clicks Start Skill Assessment, chooses a category, and starts the quiz. The program displays questions and four answer options.

3. Saving the Result

After the last question is answered, the program calculates the score and saves the category, score, total, and timestamp in the assessments table.

4. Viewing Assessment History

The user opens View Assessment History. The application reads records from SQLite and shows them in a table, with the newest record first.

5. Clearing Assessment History

The user clicks Clear History. A confirmation dialog appears. If confirmed, the application deletes rows from the assessments table and refreshes the visible table. If cancelled, the records remain.

6. Managing Practice Tasks

The user can add a task, select a task to toggle completion, or select a task and confirm deletion.

7. Viewing Recommendations

The application retrieves assessment records, selects the latest result per category, calculates the percentage, and displays advice based on the result range.

8. Algorithms Used

1. Sequential Question Processing

Questions are stored in category-based lists. The program uses an index to display each question in order and moves to the next question after an answer is selected.

2. Score Calculation

The program compares each selected option with the correct option index and increments the score when the answer is correct. The final score is displayed as score/total.

3. Database Insert and Retrieval

SQLite INSERT operations store assessment results and new tasks. SELECT operations retrieve saved assessment records and tasks.

4. Latest-Record Selection

Assessment records are retrieved in descending ID order. The recommendation module keeps the first record encountered for each category, representing that category's latest saved result.

5. Percentage and Threshold Check

The recommendation percentage is calculated as (score / total) x 100. Results below 50% receive a fundamentals priority, results below 80% receive a regular-practice priority, and other results receive an advanced-practice priority.

6. Confirmation Before Deletion

The Clear History function asks for confirmation before deleting rows from the assessments table. The tasks table is not changed by this operation.

9. Project Code

The project consists of two Python files. main.py defines the graphical interface and application flow. database.py manages SQLite storage. The complete application code used for this report is included below.

9.1 database.py

```
import sqlite3

DATABASE_NAME = "placeready.db"


def connect_db():
    return sqlite3.connect(DATABASE_NAME)


def create_tables():
    connection = connect_db()
    cursor = connection.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS assessments (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            category TEXT NOT NULL,
            score INTEGER NOT NULL,
            total INTEGER NOT NULL,
            date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            task_name TEXT NOT NULL,
            completed INTEGER DEFAULT 0
        )
    """)

    connection.commit()
    connection.close()


def save_assessment(category, score, total):
    connection = connect_db()
    cursor = connection.cursor()
    cursor.execute("""
        INSERT INTO assessments (category, score, total)
        VALUES (?, ?, ?)
    """, (category, score, total))
    connection.commit()
    connection.close()


def get_assessments():
    connection = connect_db()
    cursor = connection.cursor()
    cursor.execute("""
        SELECT category, score, total, date
        FROM assessments
        ORDER BY id DESC
    """)
    records = cursor.fetchall()
    connection.close()
    return records


def clear_assessments():
    """Delete assessment history without deleting practice tasks."""
    connection = connect_db()
    cursor = connection.cursor()
    cursor.execute("DELETE FROM assessments")
    connection.commit()
    connection.close()


def add_task(task_name):
    connection = connect_db()
    cursor = connection.cursor()
    cursor.execute(
        "INSERT INTO tasks (task_name) VALUES (?)",
        (task_name,)
    )
    connection.commit()
    connection.close()


def get_tasks():
    connection = connect_db()
    cursor = connection.cursor()
    cursor.execute("""
        SELECT id, task_name, completed
        FROM tasks
        ORDER BY id
    """)
    records = cursor.fetchall()
    connection.close()
    return records


def update_task(task_id, completed):
    connection = connect_db()
    cursor = connection.cursor()
    cursor.execute("""
        UPDATE tasks
        SET completed = ?
        WHERE id = ?
    """, (1 if completed else 0, task_id))
    connection.commit()
    connection.close()
def delete_task(task_id):
```

```
    connection = connect_db()
    cursor = connection.cursor()
    cursor.execute(
        "DELETE FROM tasks WHERE id = ?", (task_id,)
    )
    connection.commit()
    connection.close()


if __name__ == "__main__":
    create_tables()
    print("PlaceReady database created successfully!")
```

9.2 main.py

```
import tkinter as tk
from tkinter import ttk, messagebox
import database

# PLACEREADY PYTHON EDITION
root = tk.Tk()
root.title("PlaceReady - Smarter Placement Journey")
root.geometry("1000x650")
root.minsize(800, 550)
root.configure(bg="#f3f6fb")
database.create_tables()

# Header
header = tk.Frame(root, bg="#23395d", height=100)
header.pack(fill="x")
header.pack_propagate(False)

tk.Label(header, text="PlaceReady", font=("Arial", 26, "bold"),
         fg="white", bg="#23395d").pack(pady=(15, 0))
tk.Label(header, text="Smarter Placement Journey", font=("Arial", 12),
         fg="#dce6f5", bg="#23395d").pack()

# Main dashboard
content = tk.Frame(root, bg="#f3f6fb")
content.pack(fill="both", expand=True, padx=30, pady=25)

tk.Label(content, text="Welcome to PlaceReady!",
         font=("Arial", 22, "bold"), fg="#23395d",
         bg="#f3f6fb").pack(anchor="w", pady=(5, 10))
tk.Label(content,
         text="Prepare for placements, assess your skills, and track your progress.",
         font=("Arial", 12), fg="#4b5563",
         bg="#f3f6fb").pack(anchor="w", pady=(0, 25))

cards_frame = tk.Frame(content, bg="#f3f6fb")
cards_frame.pack(fill="x")
cards = [
    ("Coding", "Practice programming skills"),
    ("SQL", "Improve database knowledge"),
    ("Aptitude", "Develop problem-solving skills"),
    ("Communication", "Build communication skills")
]

for index, (name, detail) in enumerate(cards):
    card = tk.Frame(cards_frame, bg="white",
                    highlightbackground="#d9e2ef",
                    highlightthickness=1, padx=15, pady=20)
    card.grid(row=index // 2, column=index % 2,
              padx=10, pady=10, sticky="nsew")
    tk.Label(card, text=name, font=("Arial", 16, "bold"),
             fg="#23395d", bg="white").pack(anchor="w")
    tk.Label(card, text=detail, font=("Arial", 10),
             fg="#4b5563", bg="white",
             wraplength=300).pack(anchor="w", pady=(8, 0))

cards_frame.columnconfigure(0, weight=1)
cards_frame.columnconfigure(1, weight=1)

# Assessment question bank
question_bank = {
    "Coding": [
        ("Which data type stores a whole number in Python?", ["int", "str", "list", "dict"], 0),
        ("Which keyword defines a function?", ["class", "def", "function", "new"], 1),
        ("Which symbol is used for comments in Python?", ["//", "#", "/*", "<!--"], 1),
        ("What is the output of print(2 + 3)?", ["23", "5", "6", "Error"], 1),
        ("Which loop iterates over a sequence?", ["for", "switch", "case", "define"], 0),
        ("Which data type stores True or False?", ["int", "str", "bool", "list"], 2),
        ("Which brackets create a Python list?", ["()", "{}", "[]", "<>"], 2),
        ("Which keyword checks another condition?", ["else if", "elif", "elseif", "then"], 1),
        ("What does len([10, 20, 30]) return?", ["2", "3", "30", "10"], 1),
        ("Which keyword imports a module?", ["include", "using", "import", "require"], 2)
    ],
    "SQL": [
        ("Which command retrieves data?", ["INSERT", "SELECT", "DELETE", "UPDATE"], 1),
        ("Which clause filters records?", ["ORDER BY", "GROUP BY", "WHERE", "JOIN"], 2),
        ("Which command adds a new row?", ["INSERT", "SELECT", "DROP", "ALTER"], 0),
        ("Which command modifies existing records?", ["CHANGE", "UPDATE", "MODIFY TABLE", "EDIT"], 1),
        ("Which command removes selected rows?", ["DELETE", "SELECT", "CREATE", "INSERT"], 0),
        ("Which clause sorts query results?", ["SORT", "ORDER BY", "ARRANGE", "GROUP BY"], 1),
        ("Which function counts rows?", ["SUM()", "TOTAL()", "COUNT()", "NUMBER()"], 2),
        ("Which keyword removes duplicate results?", ["UNIQUE", "DISTINCT", "SINGLE", "DIFFERENT"], 1),
        ("Which key uniquely identifies a table row?", ["Foreign key", "Primary key", "Duplicate key", "Sort key"], 1),
        ("Which JOIN returns matching rows from both tables?", ["INNER JOIN", "CROSS JOIN", "FULL JOIN", "LEFT JOIN"], 0)
    ],
    "Aptitude": [
        ("What is 20% of 100?", ["10", "20", "30", "40"], 1),
        ("What is 12 + 18?", ["20", "25", "30", "35"], 2),
        ("What is 15% of 200?", ["15", "20", "30", "45"], 2),
        ("What is the square of 9?", ["18", "72", "81", "99"], 2),
        ("Find the next number: 2, 4, 6, 8, __", ["9", "10", "12", "14"], 1),
        ("What is the average of 10 and 20?", ["10", "15", "20", "30"], 1),
        ("If one pen costs Rs. 5, what do 4 pens cost?", ["Rs. 10", "Rs. 15", "Rs. 20", "Rs. 25"], 2),
        ("What is 3/4 expressed as a percentage?", ["25%", "50%", "75%", "80%"], 2),
        ("A car travels 60 km in 1 hour. How far in 2 hours at the same speed?", ["60 km", "90 km", "100 km", "120 km"], 3),
        ("What is 144 divided by 12?", ["10", "11", "12", "14"], 2)
    ],
    "Communication": [
        ("Which is best in a job interview?", ["Interrupting", "Listening carefully", "Ignoring questions", "Speaking
rudely"], 1),
        ("What should a professional email use?", ["Clear language", "Slang only", "No greeting", "All capital letters"], 0),
        ("What should you do if you don't understand a question?", ["Guess angrily", "Politely ask for clarification",
"Ignore it", "Leave immediately"], 1),
        ("Which is an example of active listening?", ["Interrupting", "Checking your phone", "Paying attention", "Changing
```

```
the topic"], 2),
        ("How should you begin a formal email?", ["Hey dude", "Dear Sir/Madam", "Yo", "What's up"], 1),
        ("What is important during a presentation?", ["Speaking clearly", "Speaking too fast", "Avoiding the audience",
"Reading inaudibly"], 0),
        ("Which is professional workplace behaviour?", ["Blaming everyone", "Respecting others", "Spreading rumours",
"Ignoring colleagues"], 1),
        ("How should you respond to constructive feedback?", ["Listen and learn", "Argue immediately", "Ignore everything",
"Insult the speaker"], 0),
        ("Which is suitable body language in an interview?", ["Appropriate eye contact", "Constantly looking away", "Turning
your back", "Rolling your eyes"], 0),
        ("What makes a good team member?", ["Refusing to cooperate", "Sharing ideas respectfully", "Taking all the credit",
"Ignoring teammates"], 1)
    ]
}

def open_assessment():
    window = tk.Toplevel(root)
    window.title("PlaceReady - Skill Assessment")
    window.geometry("650x450")
    window.configure(bg="#f3f6fb")

    category_var = tk.StringVar(value="Coding")
    answer_var = tk.IntVar(value=-1)
    current = {"index": 0, "score": 0, "questions": []}

    tk.Label(window, text="Skill Assessment",
             font=("Arial", 20, "bold"),
             bg="#f3f6fb", fg="#23395d").pack(pady=15)
    tk.Label(window, text="Choose a skill:", font=("Arial", 11),
             bg="#f3f6fb").pack()

    category_box = ttk.Combobox(
        window, textvariable=category_var,
        values=list(question_bank.keys()), state="readonly", width=25
    )
    category_box.pack(pady=8)

    question_label = tk.Label(window, text="",
                              font=("Arial", 13, "bold"),
                              bg="#f3f6fb", wraplength=550)
    question_label.pack(pady=20)
    options_frame = tk.Frame(window, bg="#f3f6fb")
    options_frame.pack(anchor="w", padx=70)
    feedback = tk.Label(window, text="", bg="#f3f6fb",
                        font=("Arial", 10))
    feedback.pack(pady=8)
    option_buttons = []

    for i in range(4):
        button = tk.Radiobutton(
            options_frame, text="", variable=answer_var, value=i,
            font=("Arial", 11), bg="#f3f6fb", anchor="w"
        )
        button.pack(anchor="w", pady=3)
        option_buttons.append(button)

    def show_question():
        questions = current["questions"]
        index = current["index"]
        question, options, correct = questions[index]
        question_label.config(
            text=f"Question {index + 1} of {len(questions)}\n\n{question}"
        )
        answer_var.set(-1)
        feedback.config(text="")
        for i, option in enumerate(options):
            option_buttons[i].config(text=option)

    def start_quiz():
        current["index"] = 0
        current["score"] = 0
        current["questions"] = question_bank[category_var.get()]
        category_box.config(state="disabled")
        start_button.config(state="disabled")
        next_button.config(text="Submit Answer", state="normal")
        show_question()

    def next_question():
        if answer_var.get() == -1:
            messagebox.showwarning(
                "Select an answer", "Please select an answer first.",
                parent=window
            )
            return
        questions = current["questions"]
        index = current["index"]
        if answer_var.get() == questions[index][2]:
            current["score"] += 1
        if index < len(questions) - 1:
            current["index"] += 1
            show_question()
        else:
            score = current["score"]
            total = len(questions)
            category = category_var.get()
            database.save_assessment(category, score, total)
            messagebox.showinfo(
                "Assessment Complete",
                f"Skill: {category}\nScore: {score}/{total}\nYour result has been saved!",
                parent=window
            )
            window.destroy()

    start_button = tk.Button(
        window, text="Start Assessment", command=start_quiz,
        bg="#23395d", fg="white", font=("Arial", 11, "bold"),
```

```
        padx=12, pady=6
    )
    start_button.pack(pady=5)
    next_button = tk.Button(
        window, text="Next", command=next_question,
        bg="#287a50", fg="white", font=("Arial", 11, "bold"),
        padx=20, pady=6, state="disabled"
    )
    next_button.pack(pady=5)
    question_label.config(text="Choose a category and click Start Assessment.")

def open_history():
    history_window = tk.Toplevel(root)
    history_window.title("PlaceReady - Assessment History")
    history_window.geometry("700x400")
    history_window.configure(bg="#f3f6fb")

    tk.Label(history_window, text="Assessment History",
             font=("Arial", 20, "bold"), fg="#23395d",
             bg="#f3f6fb").pack(pady=15)
    columns = ("category", "score", "date")
    table = ttk.Treeview(history_window, columns=columns,
                         show="headings", height=12)
    table.heading("category", text="Skill")
    table.heading("score", text="Score")
    table.heading("date", text="Date")
    table.column("category", width=180)
    table.column("score", width=120)
    table.column("date", width=300)
    table.pack(fill="both", expand=True, padx=20, pady=(5, 5))

    def refresh_history():
        for item in table.get_children():
            table.delete(item)
        records = database.get_assessments()
        for category, score, total, date in records:
            table.insert("", "end", values=(category, f"{score}/{total}", date))
        return records

    def clear_history():
        records = database.get_assessments()
        if not records:
            messagebox.showinfo(
                "No History", "There is no assessment history to clear.",
                parent=history_window
            )
            return
        confirm = messagebox.askyesno(
            "Confirm Clear History",
            "Are you sure you want to clear all assessment history?\n\n"
            "This action cannot be undone.",
            parent=history_window
        )
        if confirm:
            database.clear_assessments()
            refresh_history()
            messagebox.showinfo(
                "History Cleared",
                "All assessment history has been cleared successfully.",
                parent=history_window
            )

    buttons_frame = tk.Frame(history_window, bg="#f3f6fb")
    buttons_frame.pack(pady=10)
    tk.Button(
        buttons_frame, text="Clear History", command=clear_history,
        bg="#b83232", fg="white", font=("Arial", 11, "bold"),
        padx=15, pady=7
    ).pack(side="left", padx=5)
    tk.Button(
        buttons_frame, text="Close", command=history_window.destroy,
        bg="#23395d", fg="white", font=("Arial", 11, "bold"),
        padx=15, pady=7
    ).pack(side="left", padx=5)

    records = refresh_history()
    if not records:
        messagebox.showinfo(
            "No History", "No assessments have been saved yet.",
            parent=history_window
        )

def open_tasks():
    task_window = tk.Toplevel(root)
    task_window.title("PlaceReady - Practice Tasks")
    task_window.geometry("600x450")
    task_window.configure(bg="#f3f6fb")
    tk.Label(task_window, text="Practice Tasks",
             font=("Arial", 20, "bold"), fg="#23395d",
             bg="#f3f6fb").pack(pady=15)
    entry = tk.Entry(task_window, font=("Arial", 12), width=35)
    entry.pack(pady=8)
    task_list = tk.Listbox(task_window, font=("Arial", 11),
                           width=55, height=10)
    task_list.pack(pady=10)
    task_ids = []

    def refresh_tasks():
        task_list.delete(0, tk.END)
        task_ids.clear()
        for task_id, task_name, completed in database.get_tasks():
            status = "✔" if completed else "■"
            task_list.insert(tk.END, f"{status} {task_name}")
            task_ids.append(task_id)

    def add_new_task():
```

```
        task_name = entry.get().strip()
        if not task_name:
            messagebox.showwarning(
                "Missing Task", "Please enter a task.", parent=task_window
            )
            return
        database.add_task(task_name)
        entry.delete(0, tk.END)
        refresh_tasks()

    def mark_completed():
        selection = task_list.curselection()
        if not selection:
            messagebox.showwarning(
                "Select Task", "Please select a task first.",
                parent=task_window
            )
            return
        task_id = task_ids[selection[0]]
        for saved_id, task_name, completed in database.get_tasks():
            if saved_id == task_id:
                database.update_task(task_id, not completed)
                break
        refresh_tasks()

    def delete_selected_task():
        selection = task_list.curselection()
        if not selection:
            messagebox.showwarning(
                "Select Task", "Please select a task to delete.",
                parent=task_window
            )
            return
        task_id = task_ids[selection[0]]
        confirm = messagebox.askyesno(
            "Confirm Delete", "Are you sure you want to delete this task?",
            parent=task_window
        )
        if confirm:
            database.delete_task(task_id)
            refresh_tasks()

    tk.Button(task_window, text="Add Task", command=add_new_task,
              bg="#23395d", fg="white", font=("Arial", 11, "bold"),
              padx=15).pack(pady=5)
    tk.Button(task_window, text="Toggle Complete", command=mark_completed,
              bg="#287a50", fg="white", font=("Arial", 11, "bold"),
              padx=15).pack(pady=5)
    tk.Button(task_window, text="Delete Selected Task",
              command=delete_selected_task, bg="#b83232", fg="white",
              font=("Arial", 11, "bold"), padx=15).pack(pady=5)
    refresh_tasks()

def open_recommendations():
    window = tk.Toplevel(root)
    window.title("PlaceReady - Recommendations")
    window.geometry("650x450")
    window.configure(bg="#f3f6fb")
    tk.Label(window, text="Personalized Recommendations",
             font=("Arial", 18, "bold"), fg="#23395d",
             bg="#f3f6fb").pack(pady=20)
    output = tk.Text(window, font=("Arial", 11), width=65,
                     height=15, wrap="word")
    output.pack(padx=20, pady=10, fill="both", expand=True)
    advice = {
        "Coding": "Practice Python functions, loops, lists, and problem-solving.",
        "SQL": "Revise SELECT, WHERE, JOIN, GROUP BY, and aggregate functions.",
        "Aptitude": "Practice percentages, ratios, time and work, and number series.",
        "Communication": "Improve interview answers, listening, and professional email writing."
    }
    records = database.get_assessments()
    if not records:
        output.insert(tk.END,
                      "Complete a skill assessment to receive recommendations.")
    else:
        latest = {}
        for category, score, total, date in records:
            if category not in latest:
                latest[category] = (score, total)
        for category, (score, total) in latest.items():
            percentage = score / total * 100 if total else 0
            output.insert(tk.END,
                          f"{category}: {score}/{total} ({percentage:.0f}%)\n")
            if percentage < 50:
                output.insert(tk.END,
                              "Priority: Focus on the fundamentals.\n")
            elif percentage < 80:
                output.insert(tk.END,
                              "Priority: Practise regularly to improve.\n")
            else:
                output.insert(tk.END,
                              "Priority: Keep practising advanced questions.\n")
            output.insert(tk.END, advice[category] + "\n\n")
    output.config(state="disabled")

# Dashboard buttons
tk.Button(content, text="Start Skill Assessment",
          command=open_assessment, bg="#287a50", fg="white",
          font=("Arial", 12, "bold"), padx=20, pady=10).pack(pady=20)
tk.Button(content, text="View Assessment History",
          command=open_history, bg="#23395d", fg="white",
          font=("Arial", 11, "bold"), padx=15, pady=8).pack(pady=5)
tk.Button(content, text="Practice Tasks", command=open_tasks,
          bg="#6b4ca5", fg="white", font=("Arial", 11, "bold"),
          padx=15, pady=8).pack(pady=5)
tk.Button(content, text="View Recommendations",
```

```
          command=open_recommendations, bg="#c07825", fg="white",
          font=("Arial", 11, "bold"), padx=15, pady=8).pack(pady=5)

# Status bar
tk.Label(root, text="PlaceReady Python Edition | Ready to begin",
         font=("Arial", 10), fg="#4b5563", bg="#e5ebf4",
         anchor="w", padx=15, pady=10).pack(fill="x", side="bottom")

root.mainloop()
```

10. Output Screenshot

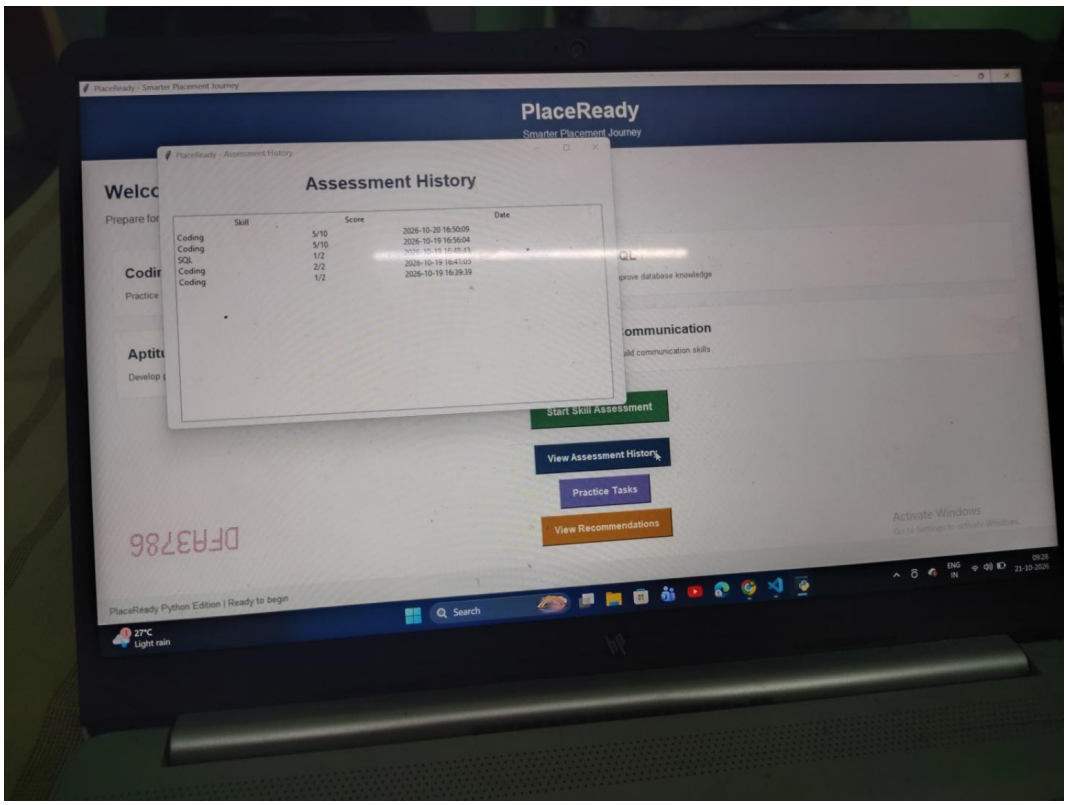


Figure 1: PlaceReady dashboard with the Assessment History window open.

Expected Functional Outputs

- The dashboard displays the PlaceReady title and navigation buttons.
- The Skill Assessment window presents questions and reports the final score.
- The Assessment History window displays saved skill scores and timestamps.
- Clear History requests confirmation and removes only saved assessment history.
- Practice Tasks supports adding, completing, and deleting tasks.
- Recommendations display score-based practice guidance.

11. Conclusion

PlaceReady demonstrates how Python can be used to build a desktop application that supports placement preparation. Its Tkinter interface brings together skill assessments, assessment history, practice task management, and recommendations in one application. SQLite stores assessment records and tasks so they can be accessed after the application is restarted.

The Clear History feature gives the user control over assessment records by requesting confirmation before deletion, while keeping practice tasks separate. The project provides practical experience with functions, lists, loops, conditional logic, graphical user interfaces, and database operations.

12. Future Enhancements

- Add more questions at different difficulty levels for each skill category.
- Provide charts to show progress across multiple assessment attempts.
- Add user login and separate records for multiple students.
- Allow users to edit or reorder practice tasks and assign due dates.
- Export assessment history and recommendations to CSV or PDF.

- Add timed assessments and explanations for correct answers.
- Improve the dashboard layout so all controls remain easy to access on different screen sizes.

13. References

1. Python Software Foundation, Python Documentation: https://docs.python.org/3/

2. Python Software Foundation, Tkinter — Python interface to Tcl/Tk: https://docs.python.org/3/library/tkinter.html

3. Python Software Foundation, sqlite3 — DB-API 2.0 interface for SQLite databases: https://docs.python.org/3/library/sqlite3.html

4. SQLite Documentation: https://www.sqlite.org/docs.html